Started on	Thursday, 27 August 2026, 3:15 PM
State	Finished
Completed on	Thursday, 27 August 2026, 4:10 PM
Time taken	54 mins 28 secs
Marks	5.00/5.00
Grade	**100.00** out of 100.00

Question 1 | Correct Mark 1.00 out of 1.00

The notion of a palindrome was introduced previously. In this exercise you will write a recursive function that determines whether or not a string is a palindrome. The empty string is a palindrome, as is any string containing only one character. Any longer string is a palindrome if its first and last characters match, and if the string formed by removing the first and last characters is also a palindrome.

Write a program that reads a string from the user and uses your recursive function to determine whether or not it is a palindrome. Then your program should display an appropriate message for the user.

Sample Input

malayalam

Sample Output

That was a palindrome!

Sample Input

madan

Sample Output

That is not a palindrome.

Answer: (penalty regime: 0 %)

Reset answer

```
def isPalindrome(s):
        # Base case: The empty string is a palindrome. So is a string containing only 1 character.
        if len(s) <= 1:
            return True
        # Recursive case: The string is a palindrome only if the first and last characters match, and
        # the rest of the string is a palindrome
        return s[0] == s[-1] and isPalindrome(s[1:-1])

# Check whether or not a string entered by the user is a palindrome
# Read the string from the user
line = input()

# Check its status and display the result
if isPalindrome(line):
    print('That was a palindrome!')
else:
    print('That is not a palindrome.')

```

	Input	Expected	Got	
✔	malayalam	That was a palindrome!	That was a palindrome!	✔
✔	madan	That is not a palindrome.	That is not a palindrome.	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question 2 | Correct Mark 1.00 out of 1.00

Given an integer number and you have to count the digits using recursion using Python program. In this program, you will be reading an integer number and counting the total digits, using a function countDigits() which will take a number as an argument and return the count after recursion process.

Input Format: The first and only line of the input contains a single integer n

Output Format: Output a single line denoting the number of digits in n.

For example:

Test	Result
`print(countDigits(800))`	3

Answer: (penalty regime: 0 %)

Reset answer

```
def countDigits(n):
    if n == 0:
        return 0
    return countDigits(n//10) + 1
```

	Test	Expected	Got	
✔	`print(countDigits(12345))`	5	5	✔
✔	`print(countDigits(800))`	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question 3 | Correct Mark 1.00 out of 1.00

Euclid was a Greek mathematician who lived approximately 2,300 years ago. His algorithm for computing the greatest common divisor of two positive integers, a and b, is both efficient and recursive. It is outlined below:

If b is 0 then
  return a
Else
  Set c equal to the remainder when a is divided by b
  Return the greatest common divisor of b and c

Write a Recursive funtion that implements Euclid's algorithm and uses it to determine the greatest common divisor of two integers entered by the user. Test your program with some very large integers. The result will be computed quickly, even for huge numbers consisting of hundreds of digits, because Euclid's algorithm is extremely efficient.

For example:

Test	Result
`print(gcd(8, 12))`	`4`
`print(gcd(720, 1000))`	`40`

Answer: (penalty regime: 0 %)

Reset answer

```
def gcd(a,b):
    if b == 0:
        return a

    else:
        c = a % b
        return gcd(b, c)
```

	Test	Expected	Got	
✔	`print(gcd(8, 12))`	`4`	`4`	✔
✔	`print(gcd(720, 1000))`	`40`	`40`	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question 4 | Correct Mark 1.00 out of 1.00

Complete the recursive function to return Binary Equivalent of an Integer using Recursion.

Sample Test Cases

Test Case 1

Input

10

Output

1010

Test Case 2

Input

257

Output

100000001

For example:

Test	Result
`print(binayNumber(10))`	`1010`
`print(binayNumber(257))`	`100000001`

Answer: (penalty regime: 0 %)

Reset answer

```
def binayNumber(n):
    if n <= 1:
        return str(n)
    return binayNumber(n//2) + str(n % 2)
```

	Test	Expected	Got	
✔	`print(binayNumber(10))`	`1010`	`1010`	✔
✔	`print(binayNumber(257))`	`100000001`	`100000001`	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question 5 | Correct Mark 1.00 out of 1.00

Complete a Recursive Function to find if a given number N can be expressed as a sum of two prime numbers.

Note: YOU MUST OPTIMIZE the logic to find whether a number is prime or not, as very large prime numbers are provided as input. If the logic is not optimized your program will NOT get executed within the given time limit.

Input Format:

First line contains number N.

Output Format:

Return either yes or no.

Boundary Conditions / Constraints:

3 <= N <= 10^9

Example Input/Output 1:

Input:

20

Output:

yes

Input:

23

Ouput:

no

Explanation:

20 can be expressed as 17+3

23 cannot be expressed as sum of two primes

For example:

Test	Result
`print(checkPrimeSum(20))`	`yes`
`print(checkPrimeSum(23))`	`no`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPrimeSum(n):
    def is_prime(x):
        if x < 2:
            return "yes"
        for i in range(2, int(x**0.5)+1):
            if x % i == 0:
                return False
        return True
    if n % 2 == 0 or is_prime(n-2):
        return "yes"
    return "no"
```

	Test	Expected	Got	
✔	`print(checkPrimeSum(20))`	`yes`	`yes`	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.